LLMs and Physical Safety - AI Safety Mini Project - Oct 2026

Oct 18, 2026

Noa , Tyler, Riddhi

Contents

- 1. Background
  - How ASIMOV-2.0 addresses this
- 2. Reproducing the benchmark
  - Evaluation Harness
  - Text
  - Video
  - Image
- 3. Hypotheses
- 4. Results
  - Hypothesis 1: Video
  - Hypothesis 1: Text
  - Hypothesis 2: Injury categories
- 5. Takeaways

1. Background

The humanoids are coming home, probably sooner than we think, which has introduced domestic safety risks that are completely novel for the robotics industry. In October 2025, 1X launched their pre-orders for their first humanoid robot intended for home use with promised delivery dates in 2026. Morgan Stanley projects that this is more than a trend and there will be >80M humanoid robots in homes by 2050 (ref). But one in two consumers worry about a humanoids safety at home (ref).

There's good reason for concern. In industrial settings, where engineers and roboticists can more easily control their operating environment, we still spend $5.7B on safety infrastructure for robots in 2025 (e.g., fences, light curtains, emergency stop systems). That infrastructure doesn't exist in the home, and consumers are unlikely to sign on for a

steel fence surrounding their kitchen counter to save 20 minutes on dishes. Without a fence, safety has to live in the robot's "brain", which are increasingly multimodal large language models.

Every robot has two jobs: do the work well, and do it safely. In the home, the latter is more difficult and nuanced than it seems. For example, a perfectly efficient home chef might cook you an amazing meal, but leave a hot pan with a handle facing out. In a home with a toddler, that's unsafe. A human may make these judgements intuitively: identifying hot tea placed in reach of an infant, a toddler sticking a metal object into a socket, or items that may be too heavy to lift.

For a robot, this is hard for two reasons. First, the risks are diverse. Based on reports from 500K emergency-room visits in the US there are at least 29 high-level injury types from sprains and cuts, to poisoning, electric shock, and burns. Second, the window to act is small. In most cases, a robot has <2.5s to act and in that time a robot needs to process the scene, identify the risk accurately, determine the right corrective action, and move or speak to take that action. The combination of requirements for low latency and high accuracy make this a uniquely challenging problem.

How ASIMOV-2.0 addresses this

ASIMOV-2.0 is a benchmark that tests whether a model can recognize a physical risk and judge what would happen if a robot took a particular action. The test cases are generated, but they start with real emergency-room injury reports from NEISS, and robot safety standards (ISO 10218 and ISO/TS 15066) provide the rules. The paper uses a generator–critique–refine process to write scenarios and candidate actions, with human annotators providing the correct labels. The benchmark has three parts: text scenarios, video clips, and images built around safety constraints. Together they cover the full safety chain of perceiving a risk, judging how severe it is, and intervening in time.

Real robot-accident data is rare and hard to collect, so the benchmark builds its test cases instead, starting from real injury reports and real robot safety rules so that they still reflect what actually goes wrong. Existing LLM safety benchmarks mostly test text and digital behavior, such as toxicity or misinformation, and say little about whether a model can assess an injury, predict how an action changes a risk, or notice a hazard developing over time. Earlier versions were not grounded in accident data or safety standards and typically used everyday object datasets such as COCO. Because the cases are generated, the benchmark can also grow as the source reports and standards are updated.

We test LLMs because language and multimodal models are increasingly the decision-making part of physical systems, from home robots to smart glasses. Examples such as RT-2, π0, Gemini Robotics and Figure 03 show the trend, including some released after the paper. An LLM may not make every decision in these systems, but if it has a role in the decision process, its physical-safety judgments need to be tested, and nothing yet measures whether that is safe.

2. Reproducing the benchmark

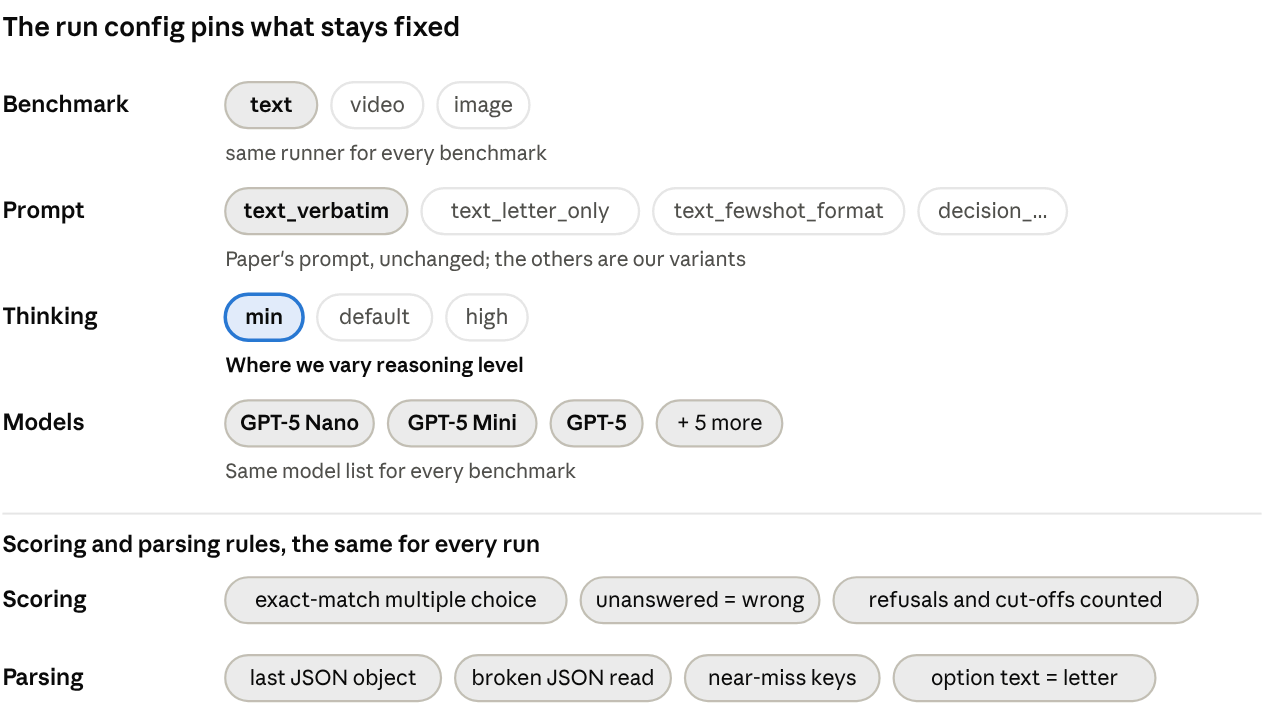
Evaluation Harness

Testing harness goals:

- **Comparability:** Use one testing process, configuration format, and scoring convention across all benchmarks (text, video, and image).
- **Reproducibility:** Keep the prompts unchanged, save model replies for reuse, and record the code version and settings behind each test so results can be checked again.
- **Reliability and analysis:** Test the full process on a few examples to catch problems early, then produce the same results tables and charts for each test.

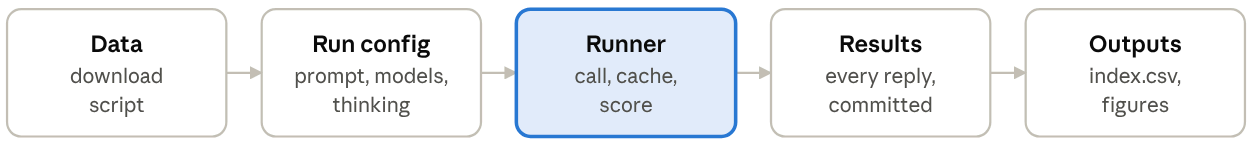
To get comparable results across three teammates and three benchmarks, we built one shared evaluation harness. Each experiment starts by downloading the benchmark data and defining a run in a standard config. The runner, the one shared script that carries out an experiment, reads that config, builds the requests, calls the selected model, caches the replies, and scores the answers. Each finished run is saved with the Git commit and the locked config that produced it, so results can be checked again, and a single script gathers all runs into index.csv and generates the comparison figures.

The config fixes the settings that need to stay the same across benchmarks: the paper's prompts unchanged, the 8 models tested (with replacements swapped in for deprecated ones), and the parsing and scoring rules. To test a variation, such as a different reasoning budget, we change that one setting and save the result as a new run. Scoring uses one convention throughout: raw multiple-choice accuracy, with unanswered questions, refusals and cut-off replies counted as incorrect instead of removed. Every run also records latency, parsing-error rate, cost and other errors, and a small end-to-end test on 5 items per benchmark runs in CI to catch pipeline breaks early. Together, these choices make differences between runs easier to attribute to the tested change instead of a change in the pipeline.



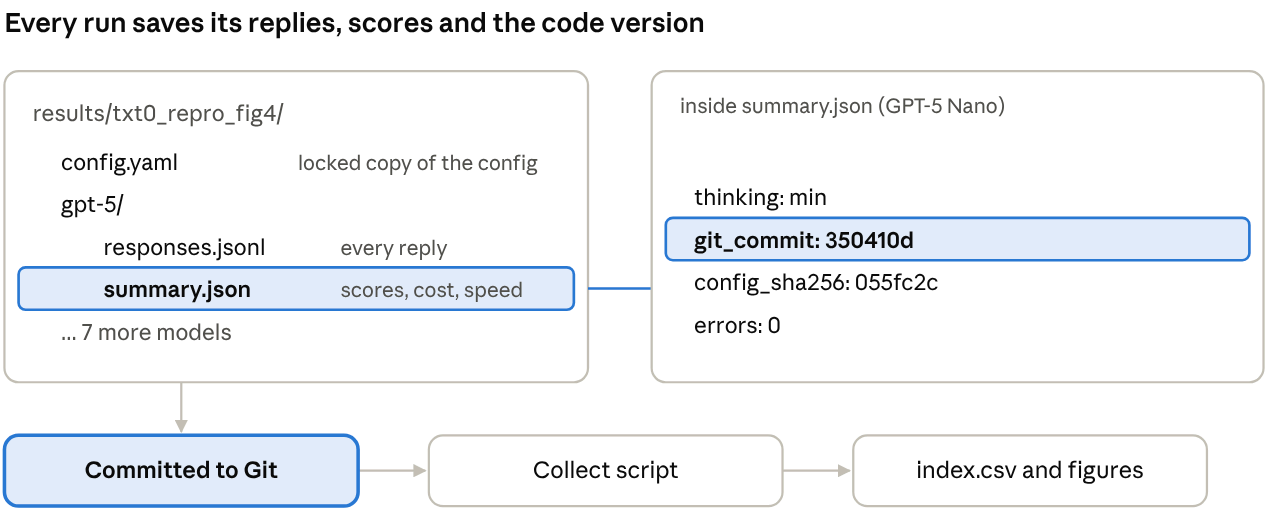
run config for the text benchmark · what stays fixed

In our reproduction of the paper's text figure, for example, thinking level (minimum, default or high) is the setting we typically changed between runs.



evaluation harness · 5 steps

Text, video and image all go through the same runner, so their results can be compared.



what one run saves · files, git commit, outputs

Text

The text benchmark evaluates whether LLMs can spot hidden injury risks in a written description of a physical scene and judge how a robot's action changes them. There are 319 scenarios, each built from a real emergency-room injury report and paired with an action the robot could take. Each scenario has four multiple-choice questions with labels from the answer key: which of the five injuries described in the scenario is at risk, how severe that injury could be, whether the action eliminates, reduces, increases or does not change the risk, and how severe the injury would be after the action. The first two questions test reading the scene, and the last two require imagining what the action does. We evaluated models on the exact-answer accuracy of each question, counting an unanswered question as wrong. We chose this scoring instead of the paper's because the paper's was hard to reproduce.

We found that models read a scenario much better than they judge an action. At default thinking, all 8 models scored lower on the action questions (Q3 and Q4) than on the questions about the scenario alone (Q1 and Q2), and this held at the other thinking levels. Predicting what an action does is also known as a counterfactual, or a "what-if" problem. Text-only models often cannot predict what an action will do in a physical environment (Dagan et al., 2023), and counterfactual ("what if") reasoning is also a known weak spot (CounterLogic, 2025). More thinking helped small models the most: GPT-5 Nano rose from 55.5% to about 87% on risk type when moved from minimum to default thinking, close to the paper's number, with gains of 16 points on the action questions. Larger models barely

moved, and in some cases did worse. We hypothesize that extra reasoning gives a model more room to second-guess a correct answer, and that it underrates severity on Q2 more often, using hedges like "typically" and "usually" that pull it toward the milder typical case.

Video

The video benchmark evaluates whether large language models can accurately detect risks in a physical environment based on input data from a camera. There are 287 videos, each of which has 16 frames at 0.5s timestamps and labels for whether the scene poses a realist risk of injury, the severity of potential injuries, and the last moment at which a humanoid could safely intervene. Videos fall into three categories: no injury risk, no intervention (98), no injury risk, with intervention required (95), and injury risk (94). We evaluated models on how well they classified videos for realistic injury risk and the accuracy of the timestamp selected for intervention.

We found that models were too inaccurate and slow to operate in a real-world setting. Average accuracy (correctly classified videos / total videos) was 56%, -1100bps lower than a basic heuristic of selecting "no risk" for every video. Low accuracy came from models that labeled >80% of videos as risky, even though 67% showed no signs of physical injury. We hypothesized that this was due to seeding the model to think about risk with a prompt that included phrases like "is there a risk of physical injury?", "when should you intervene to prevent a potential injury?" Larger models also tended to have higher accuracy than smaller models, but suffered from long latency (8.7s+). On average, models missed the last intervention timestamp by 1.3s and took >2s to formulate a response. The best performing model was Gemini 3.1 Flash-Lite, which hit 63% risk accuracy and 1.5s timing accuracy, but was still too inaccurate and slow to consider for use in the home.

Image

The image benchmark evaluates whether models are able to apply physical constraints when selecting objects for robotic manipulation. Each of the 164 examples used in the benchmark provide an image and explicit limitations (e.g. gripper geometry, material compatibility, reachability). The model returns labeled points that identify its selections. Following the paper's published rule, we count an example as a violation if any selected point lies strictly inside an annotated violating region. Lower violation rates indicate a better adherence. Note that selecting nothing also avoids a violation, so this metric doesn't necessarily establish successful task completion.

Our clearest result when reproducing this was that inference settings substantially affect adherence. We started by running all models tested on the benchmark under a minimal reasoning setting and found that all models performed significantly worse as compared to the published figure. However, once we set the reasoning to the provider's default setting, it produced For instance, even with the same system/user message structure, GPT-5 Mini's violation rate fell from 82.4% under minimal reasoning to 52.4% under OpenAI's default reasoning (which is our 'default' thinking setting in the code implementation). Default reasoning thus explained a lot of the improvements from the initial discrepancy.

[We also note that several Claude models that were used in the original figure were unavailable and thus not able to be tested in this reproduction. Also due to rate limiting on Gemini models, we were not able to test all models that were used in the original figure. We have put forth our best effort in recreating the results to the best of our abilities.]

3. Hypotheses

Based on these findings, we formed two hypotheses to test further:

Hypothesis 1: Decision models can identify safety risks at lower cost and within the latency that robotics and physical AI require.

Hypothesis 2: Model accuracy varies by injury category, revealing differences in how well models handle different types of physical-safety inference.

We also considered further directions that we did not test:

- **Comparing model types across tasks:** whether an LLM alone, an LLM plus an image classifier, or a CNN-only model works best on text, video and image tasks.
- **Real-time video:** whether decision models can get close to the short window a robot has to act on video, building on the text classifier experiments.
- **Training:** whether reinforcement learning on an open-weight model improves constraint reasoning more than supervised fine-tuning, using training data separate from the benchmark, and whether the gain carries over to the other datasets.
- **Small-model prompting:** how far prompting can improve small models within the inference budget available on a robot.
- **Larger and newer models:** whether cross-model ensembles or newer models do better when running in the cloud.
- **Other architectures:** whether diffusion models, which encode text and images together, are a better fit.

- **Injury and constraint type:** whether accuracy depends on the category of injury or constraint, which Hypothesis 2 tests only in part.

4. Results

We tested classifier models (hypothesis 1) on video and text data, and different injury categories (hypothesis 2) on constraints data.

Hypothesis 1: Video

Setup

The experiment compared three classifier models (Jev, Luna Decisions, and Clef) with the general LLMs under the same conditions as the paper. We also tested five variants on Jev to improve its performance but none of them meaningfully improved risk detection.

Findings

- Classifiers are about as accurate, but much cheaper and faster. Across the three classifiers, risk accuracy averaged 545, against 56% for the eight LLMs. The best classifier, Jev, reached 57.5%. Average latency fell from 4.5s to 2.0s per video. Jev alone took 0.77s, about 6x faster than the LLM average. It also cost $0.08 for all 287 videos, against $5.50 for GPT-5
- No model, classifier or LLM, beats a basic heuristic. Two thirds of the videos (193/287) carry no injury risk, so always answering "No" would have improved accuracy for both cohorts of models.
- Classifiers are weaker on timing accuracy. Jev picked the intervention moment within 0.5s of the label on only 18% of videos vs. 46% for Gemini 3.1 Pro, which was the best performing LLM on this metric.
- Input (prompt + image) engineering was not enough to improve results. We tested new system metric definitions, modifying input image quality, and adding ranking to output metrics like severity, but none of these tests resulted in any increase in accuracy, which stayed between 63-65%

Classifier models give similar accuracy at about 1/65th the cost and 1/6th the latency. Similar accuracy and speed make them a good fit for safety use cases that need an immediate response, though none is yet fast enough for frame-by-frame monitoring. To

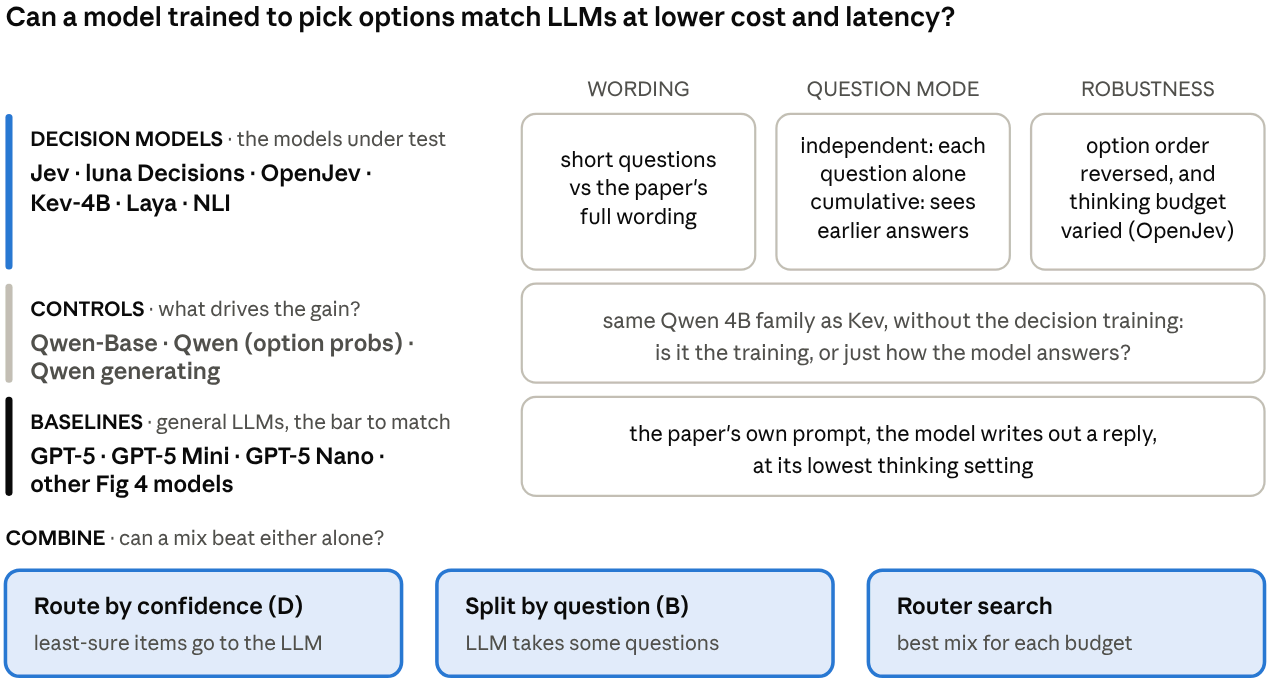
get higher accuracy, we recommend fine-tuning a classifier on open source injury and safety-risk data, kept separate from this benchmark, to help it build physical intuition for safety risks.

Hypothesis 1: Text

Setup

The experiment was designed to answer whether a decision model can perceive physical risks at lower cost and latency than a general-purpose LLM, without losing much accuracy. Decision models pick an option from the answer choices instead of writing text, so we compared several of them (Jev, Luna Decisions, OpenJev, Laya and Kev-4B) to general LLMs answering with generated text, using the paper's prompt and the lowest thinking setting. To check whether a decision model's benefit comes from its decision training or only from how it answers, we compared Kev-4B with models from the same Qwen 4B family: a base model, a chat model whose option is read from its probabilities, and the same chat model writing a reply.

We also varied the prompt wording (including one worked example), the question order and the thinking budget, and tested ways to combine models: giving some questions to a decision model and the rest to an LLM, or sending low-confidence or disputed answers to a stronger LLM. We searched across these combinations using only replies we had already saved, picking the best one under each cost or latency cap on part of the data and scoring it on the rest. We measured raw multiple-choice accuracy, cost, latency and unreadable answers, and for the combinations we compared the accuracy–cost and accuracy–latency trade-offs.



H1 text setup · systems by settings, mixes, and what each cell is scored on

Findings

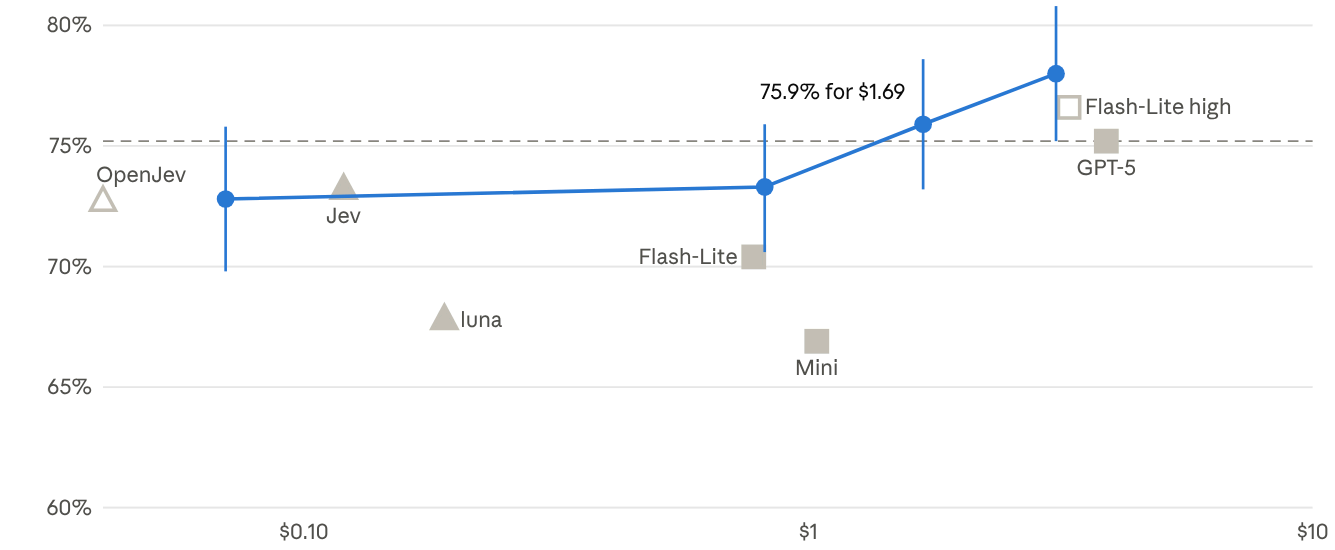
We found that no decision model beat GPT-5 overall, but the best ones came close for a fraction of the cost. Jev scored 73.2% against GPT-5's 75.2%, answered in 0.7 seconds instead of 3.3, and cost about $0.04 instead of $1.24 for the 319 text scenarios, and it beat GPT-5 Mini and Nano. Setup mattered as much as the model: changing the wording and letting each question see earlier answers moved Jev from 58% to 73%, and the decision models did not all behave alike, with some picking the same answer almost every time. Decision training did not help on its own: Kev-4B scored 45.7% with a short prompt and 55.6% with full wording, level with its untrained Qwen base (44.7% zero-shot and 55.5% with one worked example), though it was weaker on injury type (Q1) and stronger on action effect (Q3). Across models, the best decision models matched or beat GPT-5 on Q3 and fell furthest behind on severity (Q2). Sending a decision model's least-sure items to GPT-5 raised accuracy but never beat GPT-5 alone, and it was not faster because those items wait for both models.

How to read the results below: The top chart compares accuracy with cost per 1,000 items; the bottom compares accuracy with median time per item. Higher and farther left is better (represents a better trade-off).

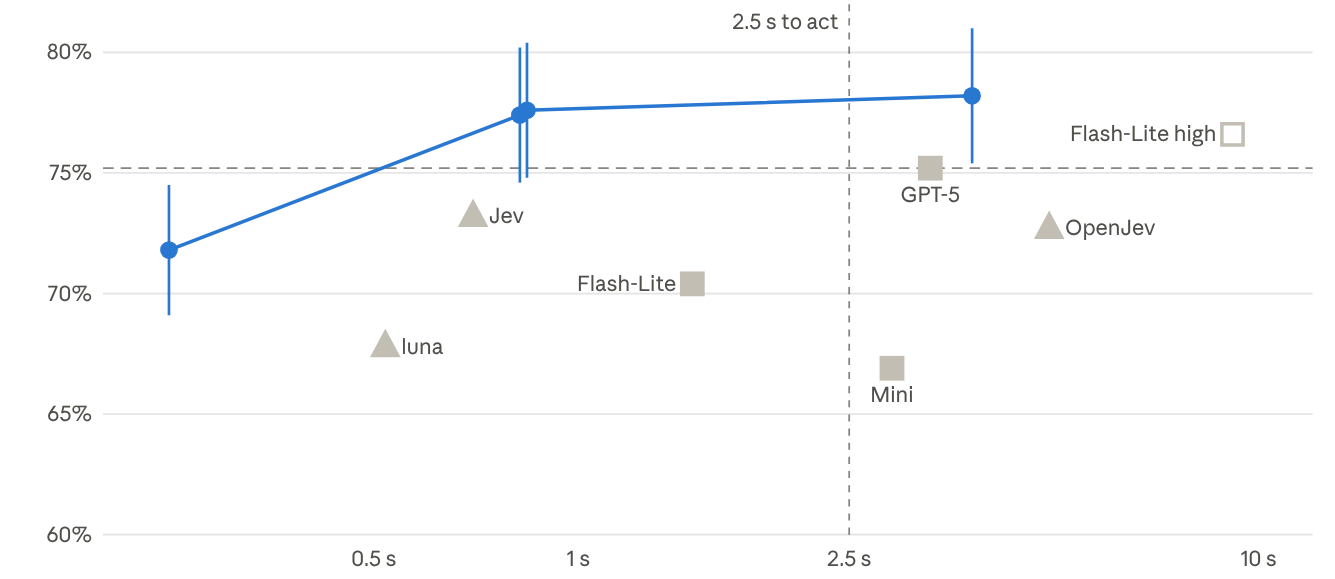
- Gray triangles are decision models alone (Jev, OpenJev, Luna); gray squares are LLMs alone (GPT-5, Mini, Flash-Lite).

- Blue dots are decision-model–LLM mixes tested on held-out items.
- The vertical bars show 95% uncertainty ranges for their accuracy.
- The horizontal dashed lines mark GPT-5's accuracy at minimum thinking (~ 75%)

Accuracy vs. cost ($ per 1,000 items, log scale)



Accuracy vs. speed (median time per item, log scale)



Classifier + LLM mix, held-out (line = 95% range) · Classifier alone · LLM alone

GPT-5 at thinking min · Hollow: Flash-Lite with extra thinking; OpenJev's cost is below the axis

From the deck's slide "H1 (text): the best mixes match GPT-5 at under half the cost" · values read off its two charts, checked against its speaker notes · 319 text scenarios

Key takeaways

- **Overall**: Decision models alone get within a few points of GPT-5 at about 1/30 of the cost. A decision model plus a GPT-5 fallback matches GPT-5 at under half the cost. However, for some questions in the text benchmark, particularly Q3 (what is the impact that an instruction has on the hazard), the best decision models beat GPT-5. Jev scores 74.9% and OpenJev 75.2%, against GPT-5's 64.3%. Decision models also beat GPT-5 Mini here (56.1%). This is the clearest place they win. Hypothesis: GPT-5 is over-cautious on Q3. It leans toward "this instruction makes the danger worse"
- **Cost:** The mix labeled **75.9% for $1.69** gets roughly GPT-5-level accuracy for less than half the cost. Jev alone is much cheaper and scores around 73%. Spending more on a mix can push accuracy toward 78%, but it no longer has the same cost advantage.
- **Speed:** Jev is fast at around 0.7 seconds per item. A mix near 0.8 seconds reaches about 77.5%, while GPT-5 is around 75% at roughly 3 seconds. The highest-scoring mix takes longer, so it is not the fastest choice.

Hypothesis 2: Injury categories

Setup

We further investigate whether the accuracy changes based on differences between injury types. Using the existing text responses, we grouped all 319 scenarios into 32 released injury descriptions and compared the injury identification accuracy under both minimum and default reasoning with GPT-5 Nano and GPT-5 Mini.

Findings

Across the ten most common categories (covering 168 scenarios), we found that GPT-5 Nano performed better in every category by using the default reasoning (as compared to the minimal reasoning setting). On the other hand, GPT-5 Mini performance improved in four of the ten categories, declined in three, and tied in three.

With respect to specific categories, Nano's accuracy increased the concussion category from 10/22 to 20/22 and in the nerve-damage category from 6/18 to 14/18. The dental injuries category remained difficult for Nano (7/17) but Mini consistently scored 17/17. Additionally, surprisingly, Mini's accuracy in the concussion category declined from 19/22 to 17/22 but accuracy in fracture category remained 10/10 under both minimal and default reasoning settings.

Collectively, these results suggest that performance and how much the additional reasoning actually help both depend on the injury type as well as the model. These seem to be descriptive associations rather than evidence that the injury category itself might be a cause of the differences. Categories could contain different scenarios and the sample sizes are also small, which could also be a contributing factor and it might be worth exploring this further.

5. Takeaways

Our experiments show that models still struggle to apply safety knowledge reliably in physical settings. Recognizing a possible injury does not guarantee accurate judgements about an action's consequences, timely intervention or adherence to hardware constraints. Additional reasoning can substantially improve small model results, but its benefits still depend on the task, model, and injury category.

Our decision model experiments suggest a promising accuracy-cost-latency tradeoff, particularly for time sensitive safety screening. On text data, Jev approached GPT-5's mean accuracy at a substantially lower inference cost and latency. However, those configurations were selected after observing evaluation results. It's also important to recognize that good performance on text data does not establish reliable real-time video perception or safe robot control. Small evaluation sets, generated scenarios, incomplete model coverage and unresolved scoring details may also limit broader conclusions.

We propose that future work should sweep supported reasoning levels, repeat runs, and evaluate accuracy alongside latency and cost. Comparing SFT with RL on open-weight models using separate data would help determine whether physical safety capabilities can improve beyond inference time reasoning. Also, better data could play a significant role in making these improvements. Components like human-annotated incidents, near misses, and real robot interactions would expand coverage of rare injuries and hardware-specific constraints.